

Guía de Arquitectura e Implementación de Código

Blueprint Estructurado para el Ecosistema CreateCard y Motor de Batalla 3D

Esta guía técnica proporciona la arquitectura de referencia, la estructura de datos y los fragmentos de código fundamentales en **JavaScript (ES6+)** necesarios para construir de forma autónoma el ecosistema interactivo de **CreateCard** y **BattleCard**. Está diseñada para ingenieros de software y diseñadores técnicos de sistemas de juego web.

1. Modelo de Datos del Objeto "Carta" (JSON Schema)

El núcleo común entre el Canvas del creador de cartas y la mesa de juego tridimensional es el estado de datos. Cada carta se modela como un objeto liso inmutable durante el ciclo de juego, validando tipos de datos estrictos:

```
{
  "id": "card_uuid_94827",
  "name": "Licantrus",           // Máx 20 caracteres
  "type": "Bestia",             // Guerrero | Mágica | Adimento | Bestia
  "level": 4,                   // Rango numérico 1 al 6
  "element": "NOX",             // GEO, AQUA, IGNIS, AER, BIO, FERRUM, NOX, FULGUR
  "atk": 1200,                  // Rango numérico 0 - 9999
  "def": 0,                     // Rango numérico 0 - 9999
  "lore": "Bestia que redunda los bosques tratando de buscar presas."
}
```

2. Motor del Creador (Módulo CreateCard)

La interfaz captura los valores modificados por el usuario a través de un formulario reactivo y renderiza sincrónicamente sobre un elemento `<canvas>` en dos dimensiones. Este fragmento demuestra cómo implementar el renderizado dinámico con el mapeo del color temático según el elemento asignado:

```

const ELEMENT_PALETTES = {
  IGNIS: '#ff4500', AQUA: '#00bfff', NOX: '#4b0082', GEO: '#8b4513'
};

function renderCardCanvas(ctx, cardData) {
  // 1. Limpiar y dibujar el fondo del contenedor de la carta
  ctx.fillStyle = '#1f2937';
  ctx.fillRect(0, 0, 350, 500);

  // 2. Aplicar color temático basado en el código criptográfico del elemento
  ctx.strokeStyle = ELEMENT_PALETTES[cardData.element] || '#ffffff';
  ctx.lineWidth = 6;
  ctx.strokeRect(10, 10, 330, 480);

  // 3. Inyección del Nombre y Atributos de Combate
  ctx.fillStyle = '#ffffff';
  ctx.font = 'bold 18px Arial';
  ctx.fillText(cardData.name.substring(0, 20), 25, 45);

  ctx.font = '14px monospace';
  ctx.fillText(`ATK: ${cardData.atk} DEF: ${cardData.def}`, 25, 450);
}

```

Uso en Producción:

Este Canvas se puede transformar instantáneamente a imagen para persistencia o descarga llamando al método nativo `canvas.toDataURL("image/png")`, guardando el string Base64 resultante directo en la base de datos del inventario del mazo del usuario.

3. Lógica del Entorno de Combate (Módulo BattleCard)

La arena controla las mecánicas por turnos. El ciclo computa la lógica matemática de daño evaluando el estado de posición del objetivo en el Slot correspondiente y altera los Puntos de Vida (**LP**) del rival en consecuencia.

```

class BattleEngine {
  constructor() {
    this.playerLP = 8000;
    this.botLP = 8000;
  }

  // Ejecución del ataque de una carta contra otra o golpe directo
  executeAttack(attackingCard, targetCard = null) {
    // Caso: Campo desprotegido -> Ataque Directo
    if (!targetCard) {
      this.botLP -= attackingCard.atk;
      return `¡Ataque Directo! El Bot pierde ${attackingCard.atk} LP.`;
    }

    // Caso: El defensor está en modo Ataque activo
    let damage = attackingCard.atk - targetCard.def;
    if (damage > 0) {
      this.botLP -= damage;
      return `Carta destruida. El Bot recibe ${damage} puntos de daño.`;
    }

    return 'La defensa fue superior. No hay alteración de LP.';
  }
}

```

4. Renderizado Espacial de la Mesa (CSS Transform Matrices)

Para simular el entorno envolvente tridimensional en la vista web de la arena de juego sin depender de WebGL o librerías externas masivas, se aplica deformación espacial tridimensional directamente sobre el nodo contenedor del tablero mediante hojas de estilo:

```

/* Aplicable al contenedor padre de la arena */
.battle-board-container {
  perspective: 1000px;
  display: block;
}

.battle-board-mesh {
  transform: rotateX(32deg) translateY(-20px);
  transform-style: preserve-3d;
  background-color: #121826;
  border: 2px solid #30363d;
  transition: transform 0.5s ease;
}

```

5. Pipeline de Integración ("Cómo Utilizar")

Para orquestar el flujo continuo donde una carta creada impacta en el flujo de combate del juego, el desarrollador debe implementar el siguiente patrón de eventos asíncronos:

1. **Fase de Creación:** El usuario finaliza la edición en el formulario. Se ejecuta la serialización llamando a una API local o actualizando un estado global reactivo.

2. **Fase de Almacenamiento en Mazo:** Se inyecta la nueva carta dentro del arreglo de objetos que componen el mazo activo del duelista: `myDeck.push(newCustomCard);`
3. **Fase de Carga en Arena:** Al inicializar el módulo **BattleEngine**, las cartas customizadas se mezclan junto al resto del arreglo homogéneo y se disponen en la pila de robo del mazo listo para la partida.